- A circuit incorporating latches has state; its output may depend not only on its current input, but also on its previous inputs. Such a circuit is described as sequential logic.
- Latch and flip-flop are a type of temporary storage device whereby they can store one bit of information.
- Latches and flip-flops are bi-stable (that has two stable states i.e. either LOW or HIGH state).
- The main difference between latches and flip-flops is the method used for changing their state.

- The output of each gate is connected to an input of the other gate
- This produces a regenerative feedback (cross-coupled)
- Can be build using
  - NOR – active HIGH
  - NAND – active LOW

Active-HIGH input S-R latch | **Active-LOW input S-R latch**

When using static gates as building blocks, the most fundamental latch is the simple SR latch, where S and R stands for 'SET' and 'RESET'. It can be constructed from a pair of cross-coupled NOR (Set/Reset) or NAND gates. The stored bit is present on the output marked Q.

Circuit (a) Active-HIGH input S-R latch :
- use NOR gate
- input R and S
- Output Q and Q'

Circuit (b) Active-LOW input S'-R' latch :
- use NAND gate
- input S' and R'
- Output Q and Q'

Circuit (c) is equivalent to circuit (b), the only difference is symbol used in the schematic. The symbol in (c) is the dual symbol of NAND gate. This symbol can represent the active-LOW input better.

Symbol for :

Example : S-R Latch (active HIGH)

State Table for active-HIGH SR Latch based on the above waveforms

Condition	S	R	Q	State
(a)	0	0	0	HOLD
(b)	1	0	1	RESET
(c)	0	0	1	HOLD
(d)	0	1	0	SET
(e)	0	0	0	HOLD
(f)	1	1	?	INV

Q4 : Do the following
a) Draw the missing output line for every signals change.
b) Name the latch used.
c) Label the output state.

Active LOW!

- Latch is a type of temporary storage device
- It has two stable states (bi-stable)
- It is level sensitive, or level-triggered.
- outputs are dependent on the voltage level applied (HIGH or LOW), not on any signal transition.

S means *Set*, R means *Reset*

INFO : A **bi-stable** state is one with two stable output states.

S-R

Gated S-R

Gated D

- S-R latch is asynchronous, which means that the output changes very soon after the input S and R change
- Gated S-R has another input called **enable** (EN), that control when the output should change
- When this input is enabled (EN =1), the output will depend on the value of S and R.
- If EN=0, the SR latch is disable, input S and R doesn't have an effect on the output, the output does not change.
- EN pin has a higher priority than pin S and R
  - SR input will only has an effect to the output if pin EN =1
  - The EN pin can be use to prevent the occurrence of invalid state at the output Q

Truth Table for Gated SR Latch operation

EN	S	R	Output Q	State
0	X	X	No Change	Disable
1	0	0	No Change	Disable
1	0	1	$Q=0$	Enable
1	1	0	$Q=1$	Enable
1	1	1	Invalid	Enable

Timing diagram that shows the effect of EN input

Q4 Draw the output Q waveform.

- For Gated S-R, introduce gate G3 and G2
  - behaves as a gate that control whether to allow S to Y and R to Z or to make Y=Z=1.
  - Y = S.EN' and Z = R.EN'
  - If EN=0, Y=Z=1
  - If EN=1, Y=S' and Z = R' this will change an active-low SR to an active HIGH SR flip-flop

Referring to logic diagram above, fill in the table. Assume initially Q = 0 and Q' = 1

EN	S	R	Y	Z	Q
0	0	0	1	1	0
0	0	1	1	1	0
0	1	0	1	1	0
0	1	1	1	1	0
1	0	0	1	1	0
1	0	1	0	1	1
1	1	0	1	0	1
1	1	1	0	1	1

Draw waveform for Q.

Logic Symbol & Logic Diagram

Logic diagram | Logic symbol

The purpose of the inverter is to ensure that the two complement of S and R will never be the same as Q, so that, we have only two conditions at the input of SR.

Condition 1: $S = 1$, Therefore $Y = 0$, $R = 1$ which make $Q = 0$
Condition 2: $R = 1$, Therefore $Y = 1$, $S = 0$ which make $Q = 1$

Just like gated SR latch ON, if the output does not change, $R=Y$, if $EN = 1$, the output will depend on the value of input D.

You can use Controller Mod $Q = D$ when $EN = 1$.

The characteristic of the gated D latch can be represented in the truth table →

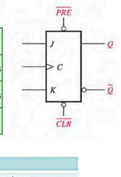
EN	D	Output Q
0	X	No Change
1	0	No Change
1	0	$Q = 0$
1	1	$Q = 1$

Draw waveform for Q.

FF Asynchronous Input

- Synchronous → inputs transferred on the triggering edge of the clock
- Asynchronous → inputs effect FF state independent of the clock
  - Normally labeled (depends on the manufacturer)
    - preset ($\overline{PRE}$) or direct set
    - clear ($\overline{CLR}$) or direct reset

$\overline{PRE}$	$\overline{CLR}$	FF	MODE
0	1	SET	Asynchronous
1	**0**	RESET	Asynchronous
1	1	JK	Synchronous



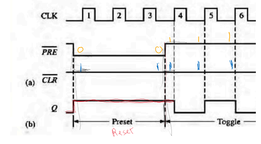
- Flip-flop input priority:

Priority	FF Input
Highest	$\overline{PRE}$, $\overline{CLR}$ - asynchronous input
Medium	Clock
Lowest	SR, JK,D,T -synchronous input

- Flip-flop asynchronous input has the highest priority.
  - Which means that if the asynchronous input active, the output will immediately change regardless the value of synchronous input or clock.
- There are two types of asynchronous input:
  - preset , $\overline{PRE}$ → to set initial value of 1 to the output
  - clear, $\overline{CLR}$ → to reset the output to 0
- Usually the asynchronous input are applied momentarily (in a short period of time), because to allow the normal operation of the FF both asynchronous input must be deactivated (HIGH) .

Example: J-

clk pulse	$\overline{PRE}$	$\overline{CLR}$	J	K	FF State	
1,2,3	0		1	1	SET	JK in
4,5,6, 7	1	1	1	1	Toggle	Sync
8,9		0	1	1	RESET	JK in

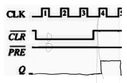


- The timing diagram is for JK Flip Flops connected as T FF, as shown in the figure →

- From the timing diagram, it can clearly illustrates the asynchronous characteristic of the $\overline{PRE}$ and $\overline{CLR}$, where when the asynchronous input change the output will immediately change without waiting for the clock edge.

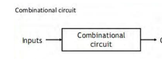
Tips:
- *If you are required to draw the timing diagram with an asynchronous input, d*
  asynchronous input first. When draw this output ignore the edge of the clock.
- *After that you can ignore the asynchronous input, draw the effect of a synchr*
  note that the output will only change at the edge of the clock.

DIY: Draw the output, Q waveform

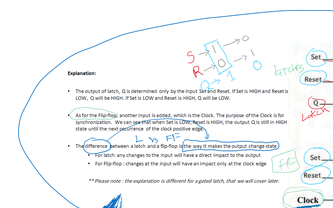


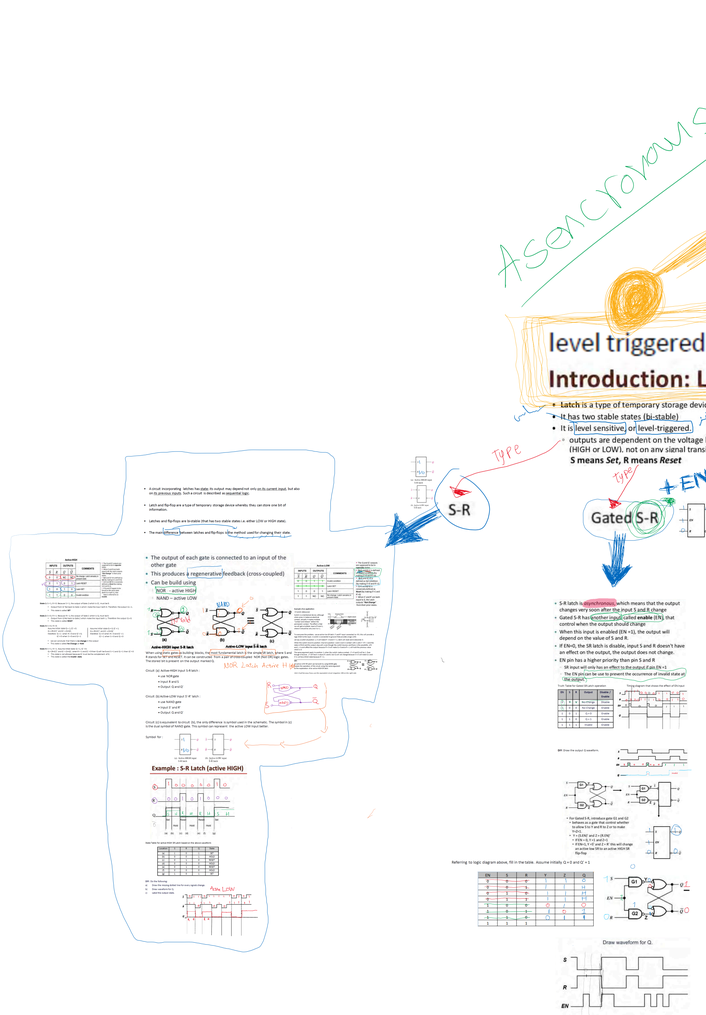
Latches

Combinational circuit



- the output depend on the input
- it has no memory element, therefore it cannot memorize the previous output

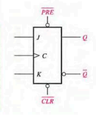




FF Asynchronous Input

- Synchronous → inputs transferred on the triggering edge of the clock
- Asynchronous → inputs effect FF state independent of the clock
  - Normally labeled (depends on the manufacturer)
    - preset ($\overline{PRE}$) or direct set
    - clear ($\overline{CLR}$) or direct reset

$\overline{PRE}$	$\overline{CLR}$	FF	MODE
0	1	SET	Asynchronous
1	**0**	RESET	Asynchronous
1	1	JK	Synchronous



- Flip-flop input priority:

Priority	FF Input
Highest	$\overline{PRE}$, $\overline{CLR}$ - asynchronous input
Medium	Clock
Lowest	SR, JK,D,T -synchronous input

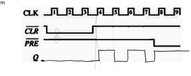
- Flip-flop asynchronous input has the highest priority.
  - Which means that if the asynchronous input active, the output will immediately change regardless the value of synchronous input or clock.
- There are two types of asynchronous input:
  - preset , $\overline{PRE}$ → to set initial value of 1 to the output
  - clear, $\overline{CLR}$ → to reset the output to 0
- Usually the asynchronous input are applied momentarily (in a short period of time), because to allow the normal operation of the FF both asynchronous input must be deactivated (HIGH) .

Example: J-K FF

clk pulse	$\overline{PRE}$	$\overline{CLR}$	J	K	FF State	Comment
1,2,3	0		1	1	SET	JK inputs - dont care
4,5,6, 7	1	1	1	1	Toggle	Synchronous mode.
8,9		0	1	1	RESET	JK inputs - dont care

- The timing diagram is for JK Flip Flops connected as T FF, as shown in the figure →

- From the timing diagram, it can clearly illustrates the asynchronous characteristic of the $\overline{PRE}$ and $\overline{CLR}$, where when the asynchronous input change the output will immediately change without waiting for the clock edge.

Tips:
- *If you are required to draw the timing diagram with an asynchronous input, draw output cause by the asynchronous input first. When draw this output ignore the edge of the clock.*
- *After that you can ignore the asynchronous input, draw the effect of a synchronous input and take note that the output will only change at the edge of the clock.*

DIY: Draw the output, Q waveform